Análise e Técnicas de Algoritmos

Aula 03

Ricardo Oliveira
ricardo@computacao.ufcg.edu.br

Tempo de execução de algoritmos

Calculando o tempo de execução de um programa

O domínio assintótico justifica a não utilização de constantes (somando ou multiplicando) na determinação da complexidade de um algoritmo.

O importante é determinar o termo mais significativo.

- Ex: $T(n) = 5n^3 + 20n^2 + 100n + 500 \Rightarrow \Theta(n^3)$

Calculando o tempo de execução de um programa

Por isso, ao contarmos operações primitivas, algumas podem ser desprezadas.

- O importante é detectar operações que **variam em função da entrada**.

Há regras para facilitar o acúmulo de operações em um algoritmo.

Calculando o tempo de execução de um programa

Operações em notação $\Theta(n)$:

1. $f(n) = \Theta(f(n))$
2. $c \times \Theta(f(n)) = \Theta(f(n))$
3. $\Theta(f(n)) + \Theta(f(n)) = \Theta(f(n))$
4. $\Theta(\Theta(f(n))) = \Theta(f(n))$
5. $\Theta(f(n)) + \Theta(g(n)) = \Theta(\max(f(n), g(n)))$
6. $\Theta(f(n))\Theta(g(n)) = \Theta(f(n)g(n))$
7. $f(n)\Theta(g(n)) = \Theta(f(n)g(n))$

Calculando o tempo de execução de um programa

Exemplo: cálculo do tempo de execução do método “bolha” de ordenação.

```
procedure bubble (var A: array [1..n] of integer);
var
    i, j, temp: integer;
begin
(1)   for i:= 1 to n-1 do
(2)       for j:= n downto i+1 do
(3)           if (A[j-1] > A[j]) then
              begin
(4)               temp:= A[j-1];
(5)               A[j-1]:= A[j];
(6)               A[j]:= temp;
              end;
end;
```

Calculando o tempo de execução de um programa

Linhas de atribuição e teste de condição requerem um tempo constante para executar, dependente de hardware e do compilador.

- Complexidade $\Theta(1)$

As linhas 4, 5 e 6 levam tempo $\Theta(1)$.

Pela regra da soma, a execução das três leva:

- $\Theta(\max(1,1,1)) = \Theta(1)$

Calculando o tempo de execução de um programa

O teste da linha 3 também tem tempo de execução constante ($\Theta(1)$).

Para medir o tempo de execução do pior caso, supõe-se que o bloco da condição sempre será executado.

- Todo o bloco tem complexidade $\Theta(1)$.

Calculando o tempo de execução de um programa

Prosseguindo, deve-se analisar os dois loops.

- Inicialmente o mais interno.

Um loop tem a complexidade do seu corpo multiplicada pelo número de vezes que é executado.

- O loop da linha 2 é executado n – i vezes.
- Pela regra da multiplicação, $\Theta((n-i) \times 1) = \Theta(n-i)$.

Calculando o tempo de execução de um programa

Para o loop externo (linha 1):

- É executado n – 1 vezes.

$$\Theta\left(\sum_{i=1}^{n-1}(n-i)\right) = \Theta\left(\frac{n(n-1)}{2}\right) = \Theta\left(\frac{n^2}{2} - \frac{n}{2}\right) = \Theta(n^2)$$

A complexidade do algoritmo bolha é **$\Theta(n^2)$**.

Regras gerais

O tempo de execução de atribuições, leitura e escrita normalmente é O(1).

O tempo de uma seqüência de comandos é dado pela regra da adição, ou seja, o maior tempo de execução entre as instruções da seqüência.

O tempo de um if é o custo das instruções executadas mais o custo para avaliar a condição. O tempo para avaliar a condição geralmente é O(1).

Regras gerais

O tempo de um **if-then-else** é o custo para avaliar a condição mais o maior dos tempos de execução entre teste verdadeiro (if) e teste falso (else).

O tempo de um loop é a soma de todos os tempos em que as instruções internas do loop são executadas.

- Negligenciando fatores constantes, é o maior tempo de execução do corpo do loop multiplicado pelo número de vezes que o loop é executado.
- É preciso somar cada loop separadamente para ter certeza.

Regras gerais

Para **loops aninhados**, analisar os mais internos.

Tempo total de execução = tempo de execução da instrução x produto do tamanho de todos os laços.

Para chamadas de **funções não recursivas**:

- É preciso contabilizar antes o tempo de execução de funções que não fazem chamadas a outras funções.
- Depois, contabilizar as funções que chamam outras funções já contabilizadas.
- O tempo de chamada de uma função é uma instrução com a complexidade que já foi calculada.

Regras gerais

Para **funções recursivas** é preciso fazer análise de recorrência.

Exercícios

Verificar o tempo de execução dos algoritmos a seguir:

```
procedure potencia (x, n: integer);
begin
    p:= 1;
    i:= 0;
    while (i < n) do
    begin
       p:= p * x;
       i:= i + 1;
    end;
end;
```

```
procedure multiplica_matriz (n:
   integer)
var
   i, j, k: integer;
begin
   for i:= 1 to n do
     for j:= 1 to n do
     begin
        C[i,j]:= 0;
        for k:= 1 to n do
            C[i,j]:= C[i,j] +
            A[i,k] * B[k,j];
     end;
end;
```

Complexidade em chamadas recursivas: Recorrências

Recorrências

Recorrências são expressões que descrevem uma função em termos dela mesma.

Ex.: $F_n = F_{n-1} + F_{n-2}$ se $n > 2$

Relações de recorrência

Tempo de execução para caso base

Condição de parada

$$T(n) = \begin{cases} c & \text{if } n = n_0 \\ a.T(f(n)) + g(n) & \text{otherwise} \end{cases}$$

Número de vezes que uma chamada recursiva é feita

Tamanho do problema resolvido pela chamada recursiva

Restante do processamento, excetuando as chamadas recursivas

Relações de recorrência

```
int fatorial (n)
    if n == 0
        return 1
    else
        return n * fatorial (n-1)
```

Relações de recorrência

```
int fatorial (n)
    if n == 0
        return 1
    else
        return n * fatorial (n-1)
```

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ T(n-1) + 1, & \text{se } n > 0 \end{cases}$$

Relações de recorrência

```
BuscaLinearRecursiva(A, n, x)
    se n == 0 então
        devolve -1
    se A[n] == x então
        devolve n
    devolve BuscaLinearRecursiva(A, n − 1, x)
```

Relações de recorrência

```
BuscaLinearRecursiva(A, n, x)
    se n == 0 então
        devolve -1
    se A[n] == x então
        devolve n
    devolve BuscaLinearRecursiva(A, n − 1, x)
```

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ T(n-1) + 1, & \text{se } n > 0 \end{cases}$$

Relações de recorrência

```
BuscaBinariaRecursiva(A, esq, dir, x)
    se esq > dir então
        devolve -1
    meio = ⌊(esq + dir) / 2⌋
    se A[meio] == x então
        devolve meio
    senão se x < A[meio] então
        devolve BuscaBinariaRecursiva(A, esq, meio − 1, x)
    senão
        devolve BuscaBinariaRecursiva(A, meio + 1, dir, x)
```

Relações de recorrência

```
BuscaBinariaRecursiva(A, esq, dir, x)
    se esq > dir então
        devolve -1
    meio = ⌊(esq + dir) / 2⌋
    se A[meio] == x então
        devolve meio
    senão se x < A[meio] então
        devolve BuscaBinariaRecursiva(A, esq, meio − 1, x)
    senão
        devolve BuscaBinariaRecursiva(A, meio + 1, dir, x)
```

$$T(n) = \begin{cases} 1, & \text{se } n \leq 1 \\ T\left(\frac{n}{2}\right) + 1, & \text{se } n > 1 \end{cases}$$

Relações de recorrência

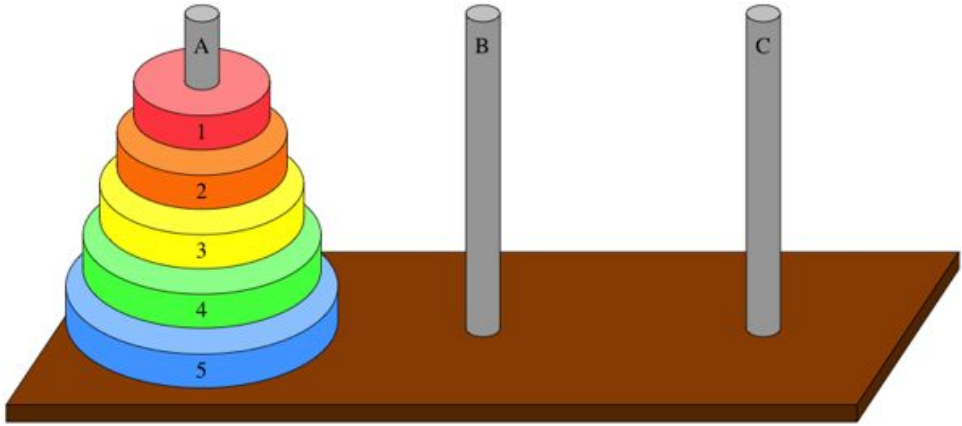
HANOI(n, orig, dest, aux)

se n > 0 então

HANOI(n - 1, orig, aux, dest)

move n de origem para destino

HANOI(n - 1, aux, dest, orig)



Relações de recorrência

```
HANOI(n, orig, dest, aux)
    se n > 0 então
        HANOI(n - 1, orig, aux, dest)
        move n de origem para destino
        HANOI(n - 1, aux, dest, orig)
```

$$T(n) = \begin{cases} 1, & \text{se } n = 1 \\ 2T(n-1) + 1, & \text{se } n > 1 \end{cases}$$

Relações de recorrência

Apesar de recorrências descreverem tempo de algoritmos recursivos de forma muito simples, as expressões

- $T(n) = T(n-1) + 1$
- $T(n) = T(n/2) + 1$
- $T(n) = 2T(n-1) + 1$

não descrevem a taxa de crescimento do algoritmo.

É preciso **resolver** a recorrência.

Exercícios

Encontre as relações de recorrência para os algoritmos:

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

```
int g(int n) {
    if (n <= 1) return 1;
    for (int i = 0; i < n; i++)
        System.out.println(i);
    return g(n/2) + g(n/2);
}
```

Complexidade em chamadas recursivas: Método Iterativo

Método iterativo

Recorrência (fatorial):

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ T(n-1) + 1, & \text{se } n > 0 \end{cases}$$

1: T(n) = T(n-1) + 1

T(n-1) = T(n-2) + 1

2: T(n) = T(n-2) + 2

T(n-2) = T(n-3) + 1

3: T(n) = T(n-3) + 3

k: T(n) = T(n-k) + k

T(0) = 1

n - k = 0

k = n

T(n) = T(n-n) + n

= T(0) + n

= 1 + n

= O(n)

Método iterativo

Recorrência (busca binária):

$$T(n) = \begin{cases} 1, & \text{se } n = 1 \\ T\left(\frac{n}{2}\right) + 1, & \text{se } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + 1 \\ &= \left[T\left(\frac{n}{4}\right) + 1\right] + 1 \\ &= T\left(\frac{n}{8}\right) + 3 \\ &\dots \\ &= T\left(\frac{n}{2^k}\right) + k \end{aligned}$$

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n \quad \Rightarrow \quad T(n) = T(1) + \log_2 n \quad \Rightarrow \quad T(n) = 1 + \log_2 n$$

$$T(n) = O(\log n)$$

Método iterativo

Recorrência (Torres de Hanoi):

$$T(n) = \begin{cases} 1, & \text{se } n = 1 \\ 2T(n-1) + 1, & \text{se } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2[2T(n-2) + 1] + 1 \\ &= 4T(n-2) + 3 \\ &= 8T(n-3) + 7 \\ &\dots \\ &= 2^k T(n-k) + (2^k - 1) \end{aligned}$$

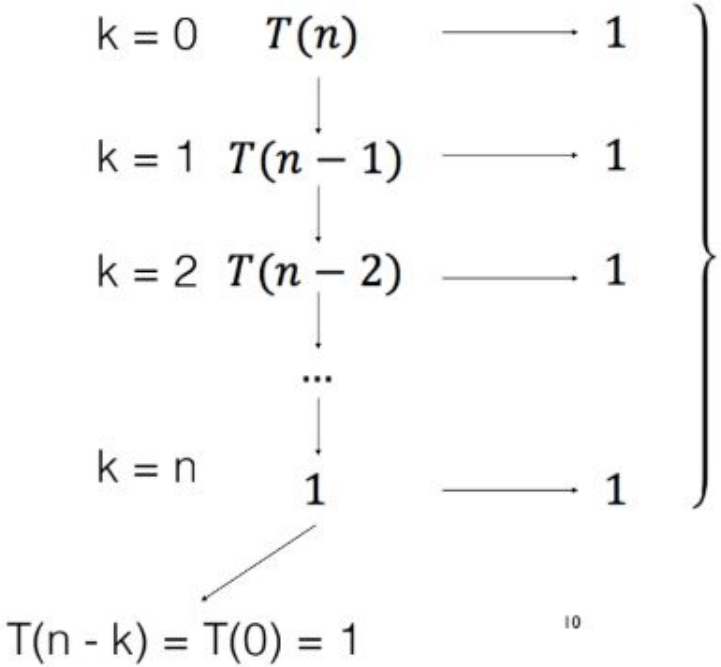
$n - k = 1 \Rightarrow k = n - 1.$ ⇨ $T(n) = 2^{n-1}T(1) + (2^{n-1} - 1)$ ⇨ $T(n) = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$

$T(n) = 2^n - 1$ ⇨ $T(n) = O(2^n)$

Complexidade em chamadas recursivas: Árvore de Recursão

Árvore de recursão

$$T(n) = \begin{cases} 1, & \text{se } n = 0 \\ T(n-1) + 1, & \text{se } n > 0 \end{cases}$$



Custo =
níveis * custo de cada nível

Custo =
(n+1) * 1 = **O(n+1)**

Mergesort

```
Algoritmo MERGESORT(A, início, fim)
    Se início < fim então
        meio ← (início + fim) / 2
        MERGESORT(A, início, meio)
        MERGESORT(A, meio + 1, fim)
        MERGE(A, início, meio, fim)
    FimSe
FimAlgoritmo
```

Mergesort

```
Algoritmo MERGE(A, início, meio, fim)
    n1 ← meio - início + 1
    n2 ← fim - meio
    Criar arrays L[1...n1], R[1...n2]

    Para i de 1 até n1 faça
        L[i] ← A[início + i - 1]
    FimPara

    Para j de 1 até n2 faça
        R[j] ← A[meio + j]
    FimPara
    i ← 1
    j ← 1

    Para k de início até fim faça
        Se i > n1 então
            A[k] ← R[j]
            j ← j + 1
        Senão se j > n2 ou L[i] ≤ R[j] então
            A[k] ← L[i]
            i ← i + 1
        Senão
            A[k] ← R[j]
            j ← j + 1
        FimSe
    FimPara
FimAlgoritmo
```

Mergesort

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{se } n > 1 \end{cases}$$

Análise e Técnicas de Algoritmos

Aula 03

Ricardo Oliveira
ricardo@computacao.ufcg.edu.br